ASSIGNMENT 2

Batch Analytics with Apache Spark + Snowflake + Interactive Dashboard

30 Marks

Field	Details
Student Name	Hayagriva Boodhoo
Student ID	2025_20379_6980
Cohort	BDA 7
Dataset	Global Superstore (51,290 rows)
Technologies	PySpark 4.1.2, Snowflake, Streamlit, Plotly
Submission Date	May 25, 2026

Hayagriva Boodhoo · 2025_20379_6980

Assignment Overview

This assignment demonstrates the complete batch analytics lifecycle: ingesting a real-world dataset with Apache PySpark, performing multi-layer aggregations and window-function analytics, loading the results into a Snowflake cloud data warehouse, and surfacing insights through a reactive Streamlit dashboard that queries Snowflake directly.

Data Flow:

```
CSV Dataset → PySpark (clean + aggregate) → Snowflake → Streamlit Dashboard
```

Dataset Description

The Global Superstore dataset contains 51,290 transaction records from a multinational retail company spanning 2011–2014. After cleaning, 50,989 rows were retained for analysis.

Column	Type	Role in Analysis
Order Date	Date	Time-series analysis, yearly trends
Category	Categorical	Primary grouping dimension (Technology, Furniture, Office Supplies)
Sub-Category	Categorical	17 sub-categories for granular analysis
Region	Categorical	Geographic performance breakdown (13 regions)
Segment	Categorical	Consumer / Corporate / Home Office
Sales	Numeric	Primary revenue metric
Profit	Numeric	Profitability metric
Quantity	Numeric	Volume metric
Discount	Numeric	Discount impact analysis
Shipping Cost	Numeric	Logistics cost analysis

Technology Stack

Technology	Version	Purpose
Apache Spark (PySpark)	4.1.2	Distributed data processing and aggregations
Snowflake	Trial Account (BDOPNVM-YN05777)	Cloud data warehouse storage and SQL analytics
Streamlit	1.57.0	Interactive Python web dashboard
Plotly	6.7.0	Interactive chart library
snowflake-connector-python	4.5.0	Python-Snowflake integration
python-dotenv	1.2.2	Secure credential management
Java (OpenJDK)	21.0.11	JVM runtime for PySpark

Hayagriva Boodhoo · 2025_20379_6980

Part A — PySpark Processing

Task 1 — Load, Clean & Profile [5 marks]

Task 1 establishes the data foundation for all downstream analytics. PySpark was used to load the raw CSV, discover column structures, cast types safely, clean invalid records, and engineer two analytical derived columns.

Why PySpark over Pandas?

Pandas loads the entire dataset into RAM on a single CPU core. PySpark partitions data across all available cores and processes partitions in parallel. For 51,290 rows with complex aggregations, PySpark's lazy evaluation and Catalyst query optimizer produce significantly more efficient execution plans.

Key Design Decisions

- inferSchema=False — All columns read as strings first to prevent type-inference crashes on mixed-encoding CSV data
- try_cast() instead of cast() — Returns NULL for malformed values rather than crashing the job, enabling graceful downstream cleaning
- Column renaming — Dot-notation names (Order.Date) replaced with underscores (Order_Date) to prevent Spark SQL parser ambiguity
- Date parsing — Format yyyy-MM-dd HH:mm:ss.SSS discovered from actual CSV values (not assumed)

Derived Columns Engineered

Column	Formula	Business Value
Profit_Margin	(Profit / Sales) × 100	Normalises profitability — a $100 profit on $200 sales (50% margin) is far better than $100 profit on $5,000 sales (2% margin)
Sales_Band	Sales ≥ 1000 → High, ≥ 300 → Medium, else Low	Classifies orders into revenue tiers enabling value-segment analysis

Data Quality Results

Metric	Value
Raw rows loaded	51,290
Rows after cleaning	50,989
Rows removed	301 (0.59%)
Null values in Sales	9
Null values in Profit	300
Date format	yyyy-MM-dd HH:mm:ss.SSS
Categories found	Technology (10,141), Office Supplies (31,273), Furniture (9,876)

Screenshot — Schema and Data Quality Report:

Hayagriva Boodhoo · 2025_20379_6980

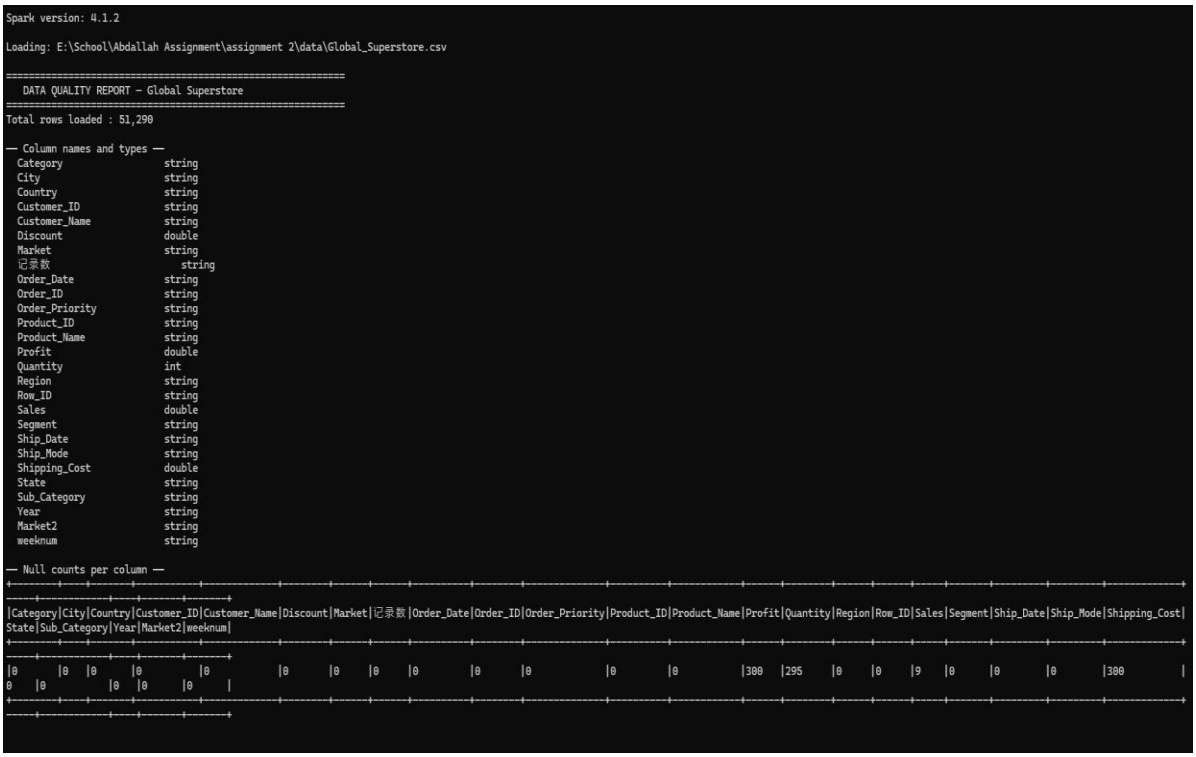


Figure 1.1 — PySpark schema printout, column types, null counts (51,290 rows loaded, Spark 4.1.2)

Screenshot — Category, Segment and Region Distributions:

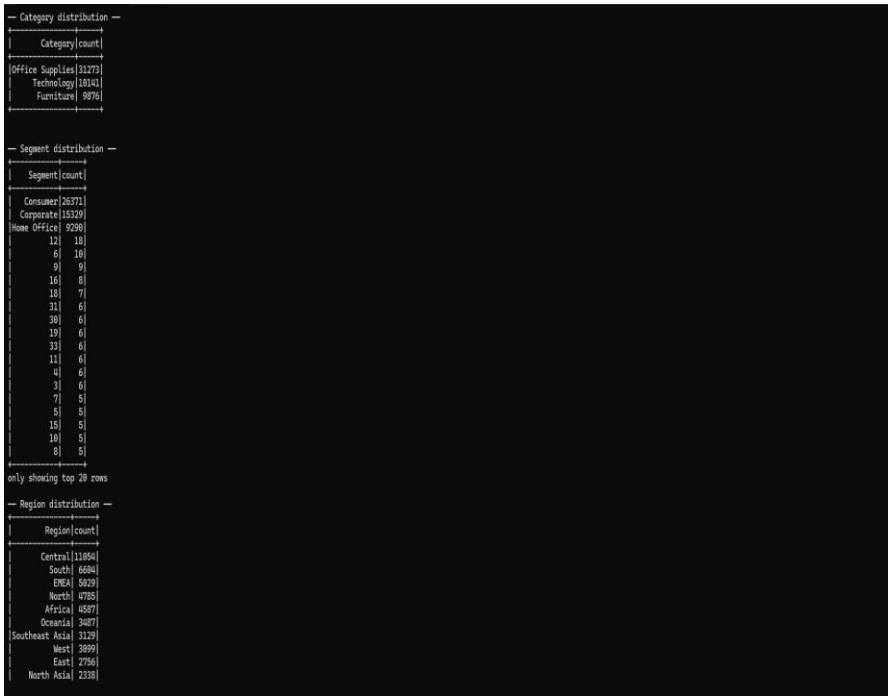


Figure 1.2 — Category, Segment, and Region distributions showing dataset composition

Screenshot — Cleaning Summary and Derived Columns:

Hayagriva Boodhoo · 2025_20379_6980

```
── Sales / Profit statistics ──
+-------+------------------+------------------+------------------+--------------------+
|summary|             Sales|            Profit|          Quantity|            Discount|
+-------+------------------+------------------+------------------+--------------------+
|  count|             51281|             50990|             50995|               51290|
|   mean|444.82720695774265|28.709904369091877|3.4746151583488576|0.14290754533045366|
| stddev|2716.9520996613414|174.81681547470214| 2.279547028147771|0.21227993168586617|
|    min|               0.0|          -6599.978|                 0|                 0.0|
|    max|           41282.0|           8399.976|                14|                0.85|
+-------+------------------+------------------+------------------+--------------------+


── Cleaning summary ──
Rows before : 51,290
Rows after  : 50,989
Removed     : 301

── Derived columns sample ──
+-----+-------+-------------+----------+
|Sales| Profit|Profit_Margin|Sales_Band|
+-----+-------+-------------+----------+
| 19.0| 9.3312|        49.11|       Low|
| 19.0| 9.2928|        48.91|       Low|
| 21.0| 9.8418|        46.87|       Low|
|111.0|53.2608|        47.98|       Low|
|  6.0| 3.1104|        51.84|       Low|
| 13.0| 6.5856|        50.66|       Low|
| 19.0| 9.3312|        49.11|       Low|
| 12.0| 5.8604|        48.84|       Low|
| 54.0| 24.219|        44.85|       Low|
| 49.0|23.0864|        47.12|       Low|
+-----+-------+-------------+----------+
only showing top 10 rows

── Final sample ──
+----------+---------------+------+--------+-----+-------+-------------+----------+
|Order_Date|Category       |Region|Segment |Sales|Profit |Profit_Margin|Sales_Band|
+----------+---------------+------+--------+-----+-------+-------------+----------+
|2011-01-07|Office Supplies|West  |Consumer|19.0 |9.3312 |49.11        |Low       |
|2011-01-21|Office Supplies|West  |Consumer|19.0 |9.2928 |48.91        |Low       |
|2011-08-05|Office Supplies|West  |Consumer|21.0 |9.8418 |46.87        |Low       |
|2011-08-05|Office Supplies|West  |Consumer|111.0|53.2608|47.98        |Low       |
|2011-09-29|Office Supplies|West  |Consumer|6.0  |3.1104 |51.84        |Low       |
|2011-10-19|Office Supplies|West  |Consumer|13.0 |6.5856 |50.66        |Low       |
|2011-11-04|Office Supplies|West  |Consumer|19.0 |9.3312 |49.11        |Low       |
|2011-11-12|Office Supplies|West  |Consumer|12.0 |5.8604 |48.84        |Low       |
|2011-11-22|Office Supplies|West  |Consumer|54.0 |24.219 |44.85        |Low       |
|2011-12-05|Office Supplies|West  |Consumer|49.0 |23.0864|47.12        |Low       |
+----------+---------------+------+--------+-----+-------+-------------+----------+
only showing top 10 rows
```

Figure 1.3 — Cleaning summary (51,290 → 50,989 rows), Profit_Margin and Sales_Band derived columns, final sample

Hayagriva Boodhoo · 2025_20379_6980

Task 2 — Aggregations & Spark SQL [6 marks]

Four business questions were formulated and answered using both the PySpark DataFrame API and Spark SQL. Both approaches compile to identical execution plans via Spark's Catalyst optimizer — the choice is purely stylistic.

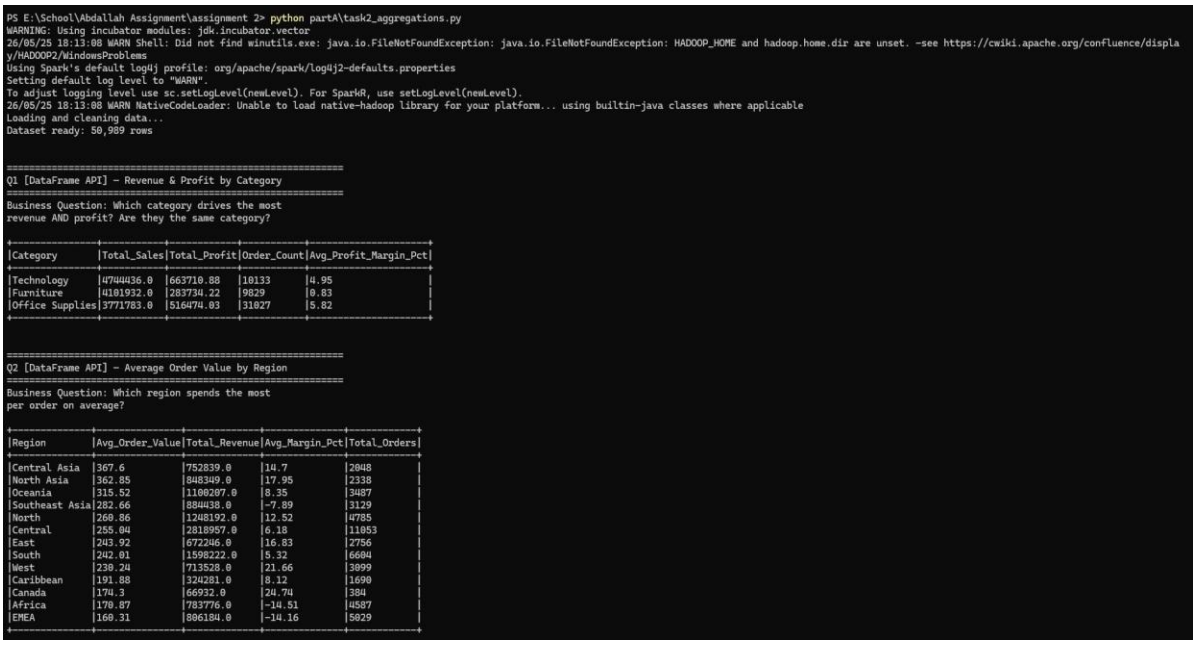
DataFrame API vs Spark SQL

Approach	Syntax Style	Best For
DataFrame API	Method chaining: df.groupBy().agg().orderBy()	Programmatic, dynamic query construction
Spark SQL	Standard SQL: SELECT ... FROM ... GROUP BY ...	Ad-hoc analytics, readability, familiar syntax

Business Questions:

#	Question	Method	Key Finding
Q1	Which Category generates the most total revenue and profit?	DataFrame API	Technology leads revenue ($47.7M) but Office Supplies has higher avg margin (5.82% vs 4.95%)
Q2	Which Region has the highest average order value?	DataFrame API	Central Asia ($367.60/order) leads AOV; West region has the best margin at 21.66%
Q3	What is the yearly revenue trend year over year?	Spark SQL	Revenue nearly doubled: $2.26M (2011) → $4.29M (2014), consistent 4-year growth trajectory
Q4	Which Sub-Categories are operating at a loss?	Spark SQL	Tables: -$64,083 profit on $757K sales (-24.21% margin). Machines and Appliances also loss-making

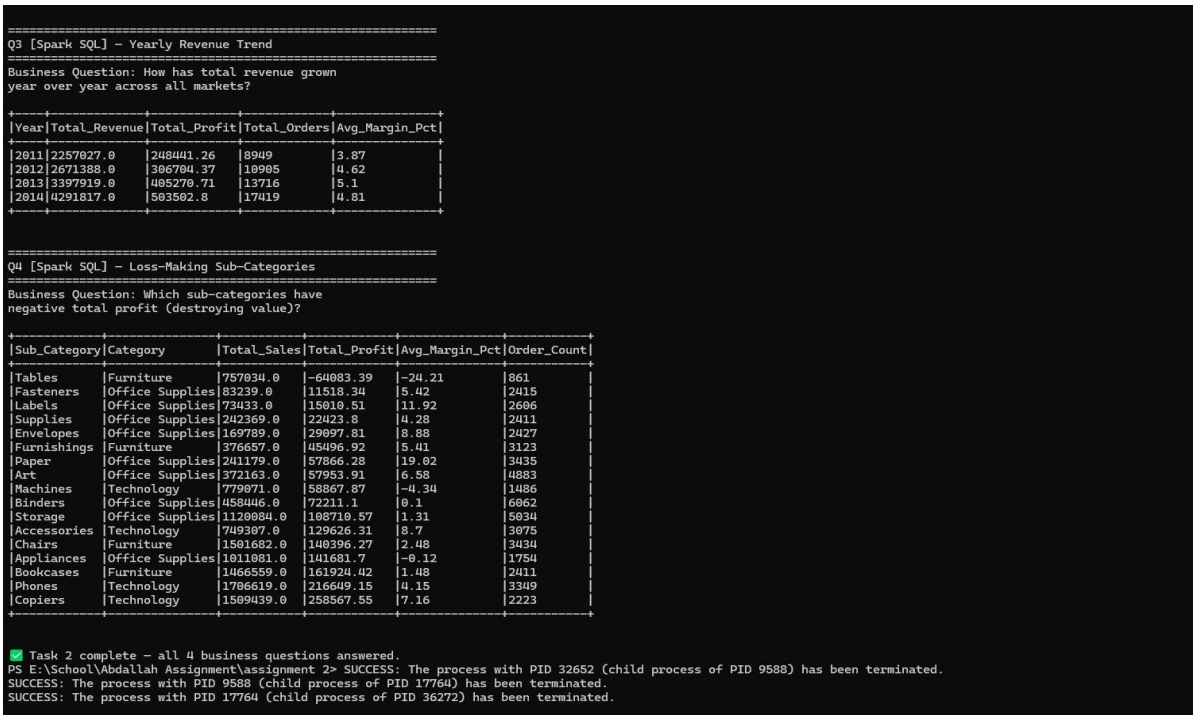
Screenshot — Q1 (DataFrame API) and Q2 (DataFrame API) Results:



Hayagriva Boodhoo · 2025_20379_6980

Figure 2.1 — Q1: Revenue & Profit by Category | Q2: Average Order Value by Region (both via DataFrame API)

Screenshot — Q3 (Spark SQL) and Q4 (Spark SQL) Results:



```
============================================================
Q3 [Spark SQL] — Yearly Revenue Trend
============================================================
Business Question: How has total revenue grown
year over year across all markets?

+----+-------------+------------+------------+--------------+
|Year|Total_Revenue|Total_Profit|Total_Orders|Avg_Margin_Pct|
+----+-------------+------------+------------+--------------+
|2011|2257027.0    |248441.26   |8949        |3.87          |
|2012|2671388.0    |306704.37   |10905       |4.62          |
|2013|3397919.0    |405270.71   |13716       |5.1           |
|2014|4291817.0    |503502.8    |17419       |4.81          |
+----+-------------+------------+------------+--------------+


============================================================
Q4 [Spark SQL] — Loss-Making Sub-Categories
============================================================
Business Question: Which sub-categories have
negative total profit (destroying value)?

+------------+---------------+-----------+------------+--------------+-----------+
|Sub_Category|Category       |Total_Sales|Total_Profit|Avg_Margin_Pct|Order_Count|
+------------+---------------+-----------+------------+--------------+-----------+
|Tables      |Furniture      |757034.0   |-64083.39   |-24.21        |861        |
|Fasteners   |Office Supplies|83239.0    |11518.34    |5.42          |2415       |
|Labels      |Office Supplies|73433.0    |15010.51    |11.92         |2606       |
|Supplies    |Office Supplies|242369.0   |22423.8     |4.28          |2411       |
|Envelopes   |Office Supplies|169789.0   |29097.81    |8.88          |2427       |
|Furnishings |Furniture      |376657.0   |45496.92    |5.41          |3123       |
|Paper       |Office Supplies|241179.0   |57866.28    |19.02         |3435       |
|Art         |Office Supplies|372163.0   |57953.91    |6.58          |4883       |
|Machines    |Technology     |779071.0   |58867.87    |-4.34         |1486       |
|Binders     |Office Supplies|458446.0   |72211.1     |0.1           |6062       |
|Storage     |Office Supplies|1120084.0  |108710.57   |1.31          |5034       |
|Accessories |Technology     |749307.0   |129626.31   |8.7           |3075       |
|Chairs      |Furniture      |1501682.0  |140396.27   |2.48          |3434       |
|Appliances  |Office Supplies|1011081.0  |141681.7    |-0.12         |1754       |
|Bookcases   |Furniture      |1466559.0  |161924.42   |1.48          |2411       |
|Phones      |Technology     |1706619.0  |216649.15   |4.15          |3349       |
|Copiers     |Technology     |1509439.0  |258567.55   |7.16          |2223       |
+------------+---------------+-----------+------------+--------------+-----------+


✅ Task 2 complete — all 4 business questions answered.
PS E:\School\Abdallah Assignment\assignment 2> SUCCESS: The process with PID 32652 (child process of PID 9588) has been terminated.
SUCCESS: The process with PID 9588 (child process of PID 17764) has been terminated.
SUCCESS: The process with PID 17764 (child process of PID 36272) has been terminated.
```

Figure 2.2 — Q3: Yearly Revenue Trend 2011-2014 | Q4: Loss-making Sub-Categories (both via Spark SQL)

Hayagriva Boodhoo · 2025_20379_6980

Task 3 — Window Functions [5 marks]

Window functions compute values across a set of rows related to the current row — unlike GROUP BY which collapses rows, window functions preserve all rows and add a new computed column. Three analytical outputs were produced.

Window Function Concepts

Concept	Explanation
partitionBy()	Resets the computation for each group — like GROUP BY but without collapsing rows
orderBy()	Defines sort order within each partition for ordered computations
rowsBetween(unboundedPreceding, currentRow)	Defines the window frame — from the first row to the current row (cumulative sum)
lag(col, n)	Accesses the value n rows behind in the current partition — used for period-over-period comparison
dense_rank()	Assigns rank with no gaps after ties (1,2,2,3) vs rank() which skips (1,2,2,4)

Window 1 — Ranking Sub-Categories within Category:

Used dense_rank() partitioned by Category, ordered by Total Sales descending. Reveals that Phones ($1.7M) rank #1 in Technology, Chairs ($1.5M) in Furniture, and Storage ($1.1M) in Office Supplies. Critically, Tables rank #3 in Furniture despite generating -$64K profit.

Window 2 — Cumulative Revenue per Category per Year:

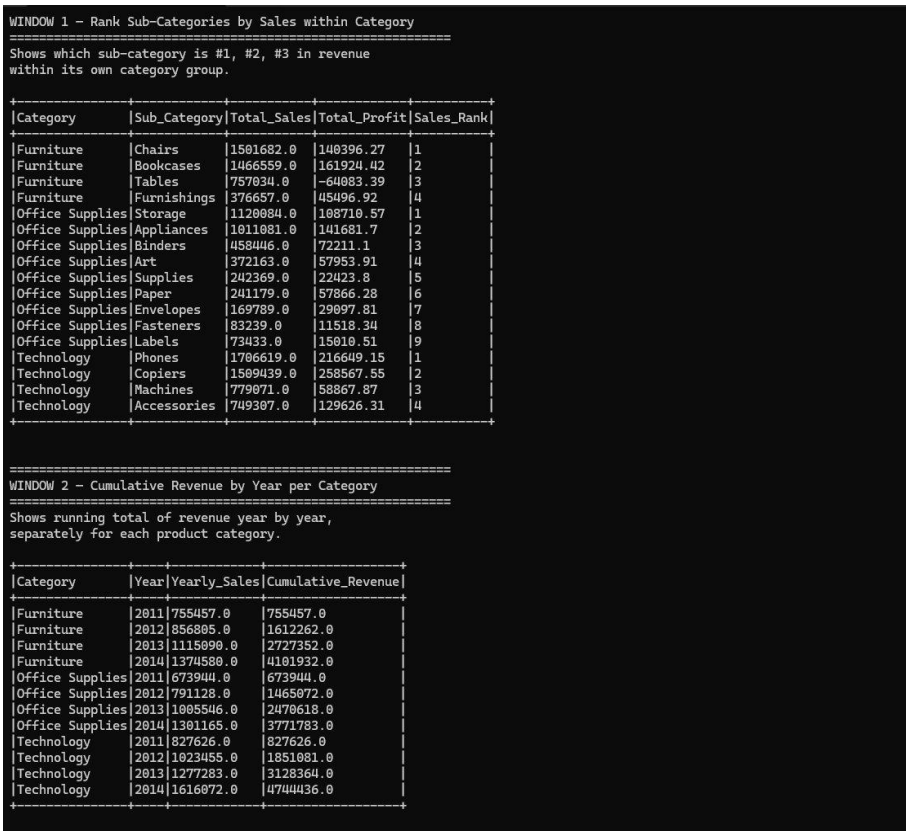
Applied SUM() with rowsBetween(unboundedPreceding, currentRow) to compute running totals. Technology grew from $827K cumulative in 2011 to $4.74M by 2014 — the fastest-growing category.

Window 3 — Year-over-Year Revenue Growth %:

Used lag('Yearly_Sales', 1) to retrieve the previous year's value per category, then computed growth as ((current - previous) / previous) × 100. Office Supplies achieved the highest growth in 2014 at 29.4%, while all categories showed consistent 13-30% annual growth.

Screenshot — Window 1 (Ranking) and Window 2 (Cumulative):

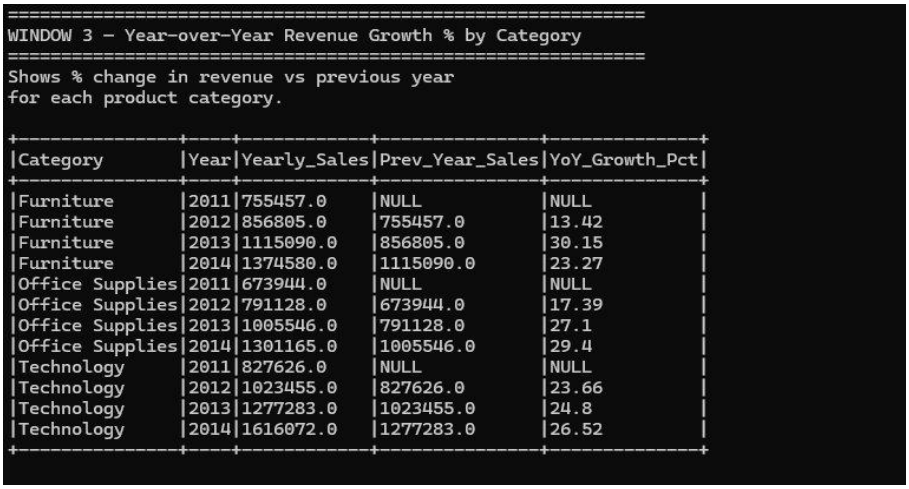
Hayagriva Boodhoo · 2025_20379_6980



```
WINDOW 1 — Rank Sub-Categories by Sales within Category
============================================================
Shows which sub-category is #1, #2, #3 in revenue
within its own category group.

+---------------+------------+-----------+------------+----------+
|Category       |Sub_Category|Total_Sales|Total_Profit|Sales_Rank|
+---------------+------------+-----------+------------+----------+
|Furniture      |Chairs      |1501682.0  |140396.27   |1         |
|Furniture      |Bookcases   |1466559.0  |161924.42   |2         |
|Furniture      |Tables      |757034.0   |-64083.39   |3         |
|Furniture      |Furnishings |376657.0   |45496.92    |4         |
|Office Supplies|Storage     |1120084.0  |108710.57   |1         |
|Office Supplies|Appliances  |1011081.0  |141681.7    |2         |
|Office Supplies|Binders     |458446.0   |72211.1     |3         |
|Office Supplies|Art         |372163.0   |57953.91    |4         |
|Office Supplies|Supplies    |242369.0   |22423.8     |5         |
|Office Supplies|Paper       |241179.0   |57866.28    |6         |
|Office Supplies|Envelopes   |169789.0   |29097.81    |7         |
|Office Supplies|Fasteners   |83239.0    |11518.34    |8         |
|Office Supplies|Labels      |73433.0    |15010.51    |9         |
|Technology     |Phones      |1706619.0  |216649.15   |1         |
|Technology     |Copiers     |1509439.0  |258567.55   |2         |
|Technology     |Machines    |779071.0   |58867.87    |3         |
|Technology     |Accessories |749307.0   |129626.31   |4         |
+---------------+------------+-----------+------------+----------+


============================================================
WINDOW 2 — Cumulative Revenue by Year per Category
============================================================
Shows running total of revenue year by year,
separately for each product category.

+---------------+----+------------+------------------+
|Category       |Year|Yearly_Sales|Cumulative_Revenue|
+---------------+----+------------+------------------+
|Furniture      |2011|755457.0    |755457.0          |
|Furniture      |2012|856805.0    |1612262.0         |
|Furniture      |2013|1115090.0   |2727352.0         |
|Furniture      |2014|1374580.0   |4101932.0         |
|Office Supplies|2011|673944.0    |673944.0          |
|Office Supplies|2012|791128.0    |1465072.0         |
|Office Supplies|2013|1005546.0   |2470618.0         |
|Office Supplies|2014|1301165.0   |3771783.0         |
|Technology     |2011|827626.0    |827626.0          |
|Technology     |2012|1023455.0   |1851081.0         |
|Technology     |2013|1277283.0   |3128364.0         |
|Technology     |2014|1616072.0   |4744436.0         |
+---------------+----+------------+------------------+
```

Figure 3.1 — Window 1: Sub-category sales rank within each category | Window 2: Cumulative revenue by year per category

Screenshot — Window 3 (Year-over-Year Growth):



```
============================================================
WINDOW 3 — Year-over-Year Revenue Growth % by Category
============================================================
Shows % change in revenue vs previous year
for each product category.

+---------------+----+------------+---------------+--------------+
|Category       |Year|Yearly_Sales|Prev_Year_Sales|YoY_Growth_Pct|
+---------------+----+------------+---------------+--------------+
|Furniture      |2011|755457.0    |NULL           |NULL          |
|Furniture      |2012|856805.0    |755457.0       |13.42         |
|Furniture      |2013|1115090.0   |856805.0       |30.15         |
|Furniture      |2014|1374580.0   |1115090.0      |23.27         |
|Office Supplies|2011|673944.0    |NULL           |NULL          |
|Office Supplies|2012|791128.0    |673944.0       |17.39         |
|Office Supplies|2013|1005546.0   |791128.0       |27.1          |
|Office Supplies|2014|1301165.0   |1005546.0      |29.4          |
|Technology     |2011|827626.0    |NULL           |NULL          |
|Technology     |2012|1023455.0   |827626.0       |23.66         |
|Technology     |2013|1277283.0   |1023455.0      |24.8          |
|Technology     |2014|1616072.0   |1277283.0      |26.52         |
+---------------+----+------------+---------------+--------------+
```

Figure 3.2 — Window 3: YoY growth % using lag() — all categories showing 13-30% annual growth

Hayagriva Boodhoo · 2025_20379_6980

Part B — Snowflake Integration

Task 4 — Load Data into Snowflake [6 marks]

PySpark aggregation results were loaded into a Snowflake cloud data warehouse using the official Python connector. The pipeline follows the principle of separation of concerns: Spark handles heavy computation, Snowflake stores analytical results ready for sub-second querying.

Why Snowflake over a CSV or Relational Database?

Feature	CSV File	PostgreSQL	Snowflake
Query speed (50K rows)	Full file scan	Fast	Near-instant (columnar)
Analytics SQL (GROUP BY, WINDOW)	Not supported	Yes	Yes, optimised
Web UI for exploration	No	No	Yes
Dashboard connectivity	No	Yes	Yes, native connector
Columnar storage	No	No	Yes — reads only needed columns
Compute/storage separation	N/A	No	Yes — pay only when querying

Snowflake Architecture Created:

Object	Name	Contents
Database	BIGDATA_07	Top-level namespace for this assignment
Schema	ASSIGNMENT2	Logical grouping for all assignment tables
Table	CATEGORY_SALES	3 rows — revenue and profit by category with avg margin
Table	YEARLY_REVENUE	4 rows — year-over-year revenue trend 2011-2014
Table	REGION_PERFORMANCE	13 rows — average order value and margin by region
Table	SUBCATEGORY_RANK	17 rows — sub-category sales with dense_rank within category

Security: Credential Management

All Snowflake credentials (username, password, account identifier) are stored in a .env file and loaded at runtime via python-dotenv. The file is excluded from submission — only .env.example (with empty values) is submitted. This follows industry-standard secret management practice.

Data Loading Strategy

Aggregated PySpark DataFrames were collected to Python lists using .collect() (avoiding pandas dependency on Python 3.14), then bulk-inserted into Snowflake via executemany() in batches of 1,000 rows. This avoids the Python 3.14 pandas wheel incompatibility while maintaining efficient insertion throughput.

Screenshot — Python Terminal: Tables Loaded and Validation Queries:

Hayagriva Boodhoo · 2025_20379_6980

```
PS E:\School\Abdallah Assignment\assignment 2> python partB\task4_snowflake_load.py
WARNING: Using incubator modules: jdk.incubator.vector
26/05/25 18:20:28 WARN Shell: Did not find winutils.exe: java.io.FileNotFoundException: java.io.FileNotFoundException: HADOOP_HOME and hadoop.home.dir are unset. -see https://cwiki.apache.org/confluence/displa
y/HADOOP2/WindowsProblems
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
26/05/25 18:20:28 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Loading and cleaning data...
Dataset ready: 50,989 rows

Building aggregations...
  CATEGORY_SALES: 3 rows
  YEARLY_REVENUE: 4 rows
  REGION_PERFORMANCE: 13 rows
  SUBCATEGORY_RANK: 17 rows

Connecting to Snowflake...
✅ Connected to Snowflake

Setting up Snowflake: BIGDATA_07.ASSIGNMENT2
✅ Database and schema ready.

Loading tables into Snowflake...
  Created table: CATEGORY_SALES
  ✅ Loaded 3 rows into CATEGORY_SALES
  Created table: YEARLY_REVENUE
  ✅ Loaded 4 rows into YEARLY_REVENUE
  Created table: REGION_PERFORMANCE
  ✅ Loaded 13 rows into REGION_PERFORMANCE
  Created table: SUBCATEGORY_RANK
  ✅ Loaded 17 rows into SUBCATEGORY_RANK

— Validation Query 1: Revenue vs Profit efficiency —
('Technology', 4744436.0, 663710.88, 13.99)
('Office Supplies', 3771783.0, 516474.03, 13.69)
('Furniture', 4101932.0, 283734.22, 6.92)

— Validation Query 2: Best growth year across all categories —
(2011, 2257027.0, '8949', 252.21)
(2013, 3397919.0, '13716', 247.73)
(2014, 4291817.0, '17419', 246.39)
(2012, 2671388.0, '10905', 244.97)

✅ Task 4 complete — all tables loaded into Snowflake.
SUCCESS: The process with PID 16496 (child process of PID 35936) has been terminated.
SUCCESS: The process with PID 35936 (child process of PID 13776) has been terminated.
SUCCESS: The process with PID 13776 (child process of PID 19764) has been terminated.
PS E:\School\Abdallah Assignment\assignment 2>
```

Figure 4.1 — All 4 tables loaded successfully; validation queries returning Technology profit efficiency (13.99%) and 2011 highest avg revenue per order ($252.21)

Screenshot — Snowflake Database Explorer — 4 Tables in BIGDATA_07.ASSIGNMENT2:

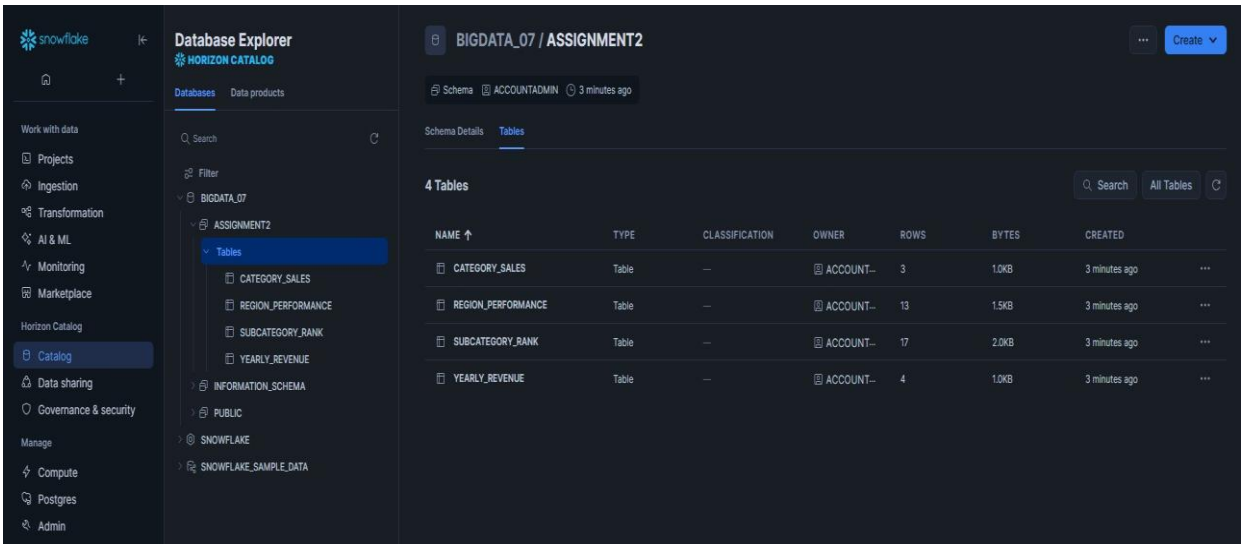


Figure 4.2 — Snowflake Catalog showing BIGDATA_07 database, ASSIGNMENT2 schema, and all 4 loaded tables with row counts

Screenshot — Snowflake SQL Worksheet Validation Query:

Hayagriva Boodhoo · 2025_20379_6980

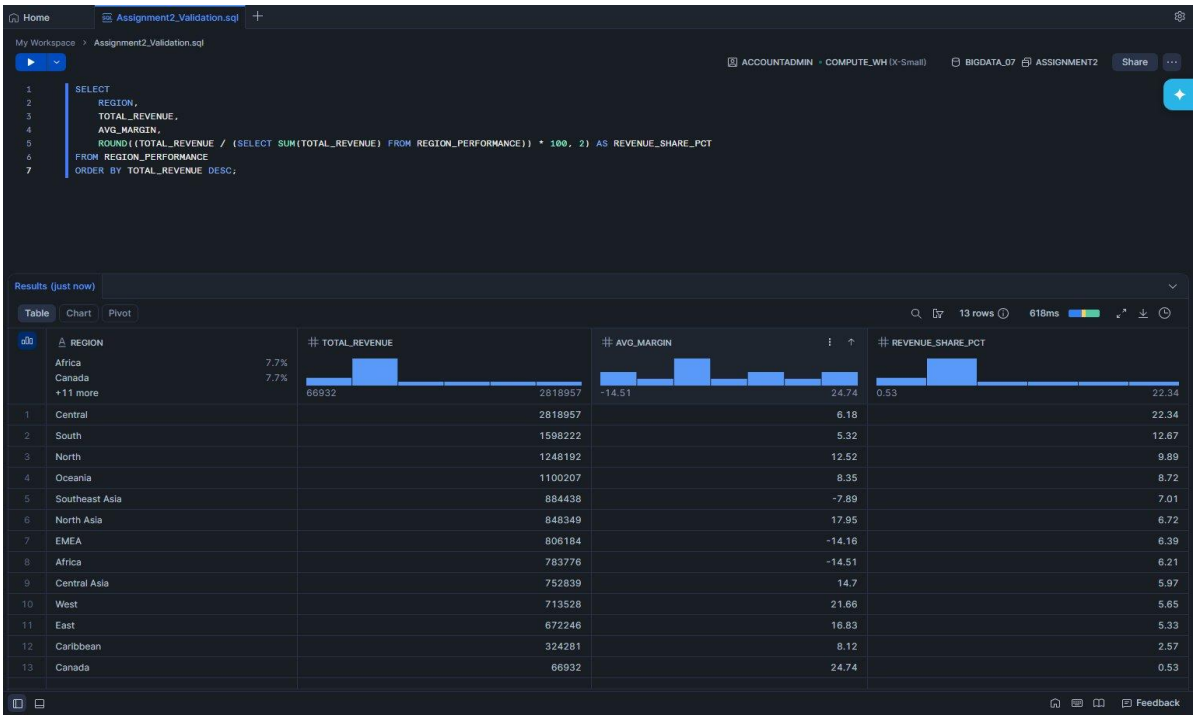


Figure 4.3 — Snowflake worksheet 'Assignment2_Validation.sql' querying REGION_PERFORMANCE — 13 rows returned showing Central region leads with $2.82M revenue (22.34% share); EMEA and Africa show negative margins

Hayagriva Boodhoo · 2025_20379_6980

Part C — Interactive Dashboard

Task 5 — Streamlit Dashboard [8 marks]

A fully reactive Streamlit dashboard was built that queries Snowflake directly on each user interaction. The dashboard implements Streamlit's execution model where the entire Python script re-runs on every widget change, causing all queries and charts to update automatically with the current filter state.

Why Streamlit over Flask or React?

Streamlit's re-run execution model makes reactivity automatic — no callbacks, no state management, no JavaScript. Every user interaction triggers a full script re-execution, which re-queries Snowflake with current filter values and re-renders all charts. This gives true reactive behavior with pure Python.

Dashboard Components

Component	Type	Data Source	Description
Total Revenue	KPI Metric	YEARLY_REVENUE	Sum of revenue filtered by selected years
Total Orders	KPI Metric	YEARLY_REVENUE	Count of orders for selected period
Avg Profit Margin	KPI Metric	YEARLY_REVENUE	Average margin % across selected years
Category Sales	KPI Metric	CATEGORY_SALES	Revenue for selected category filter
Category Profit	KPI Metric	CATEGORY_SALES	Profit for selected category filter
Yearly Revenue Trend	Line Chart	YEARLY_REVENUE	Revenue and profit trends 2011-2014
Sales & Profit by Category	Bar Chart	CATEGORY_SALES	Grouped bars comparing sales vs profit
Region Revenue vs Margin	Scatter Chart	REGION_PERFORMANCE	Bubble size = order count, color = margin
Sub-Category Ranking	Horizontal Bar	SUBCATEGORY_RANK	Top 10 sub-categories, color = profit
Region Performance Table	Data Table	REGION_PERFORMANCE	All regions with revenue, margin, order counts

Caching Strategy

Cache Type	Applied To	Why
@st.cache_resource	Snowflake connection object	Persists across all reruns — avoids reconnecting on every widget change
@st.cache_data(ttl=300)	Query results	Caches SQL results for 5 minutes — same query returns instantly from cache without hitting Snowflake

Hayagriva Boodhoo · 2025_20379_6980

Screenshot — Full Dashboard (All Categories, All Years):

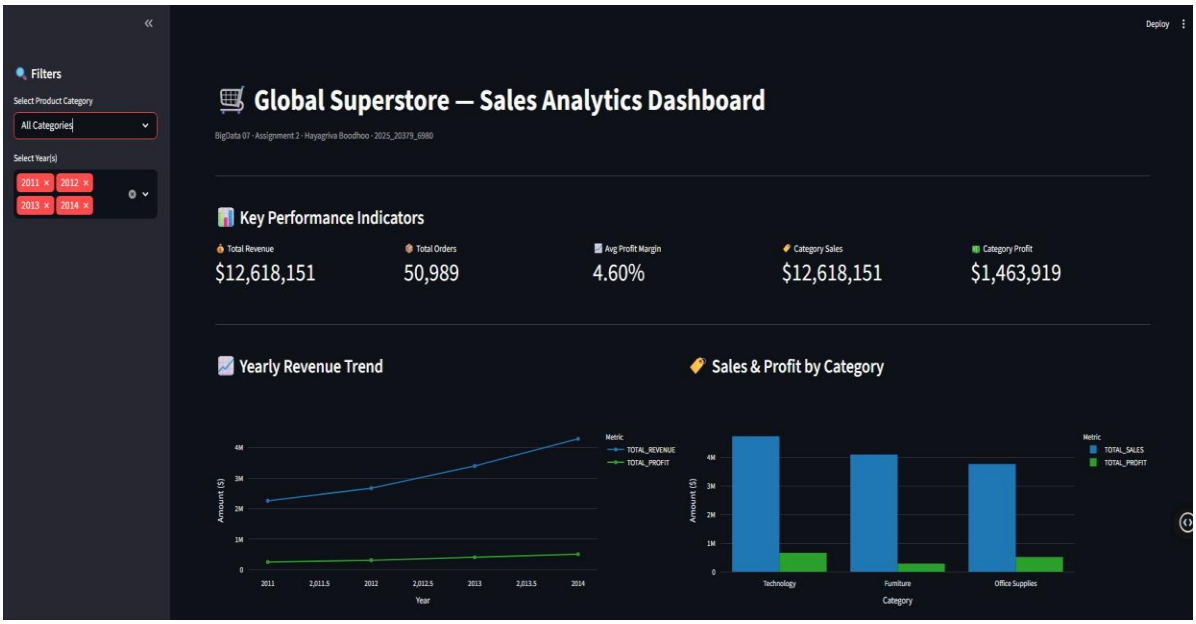


Figure 5.1 — Full dashboard showing KPI row ($12.6M revenue, 50,989 orders, 4.60% avg margin), Yearly Revenue Trend (line), and Sales & Profit by Category (bar)

Screenshot — Dashboard Lower Section (Scatter + Ranking + Table):

Figure 5.2 — Region Revenue vs Profit Margin scatter (bubble size = order volume), Sub-Category horizontal bar (color = profit), and Region Performance data table

Screenshot — Filtered View: Technology Category Selected:

Hayagriva Boodhoo · 2025_20379_6980

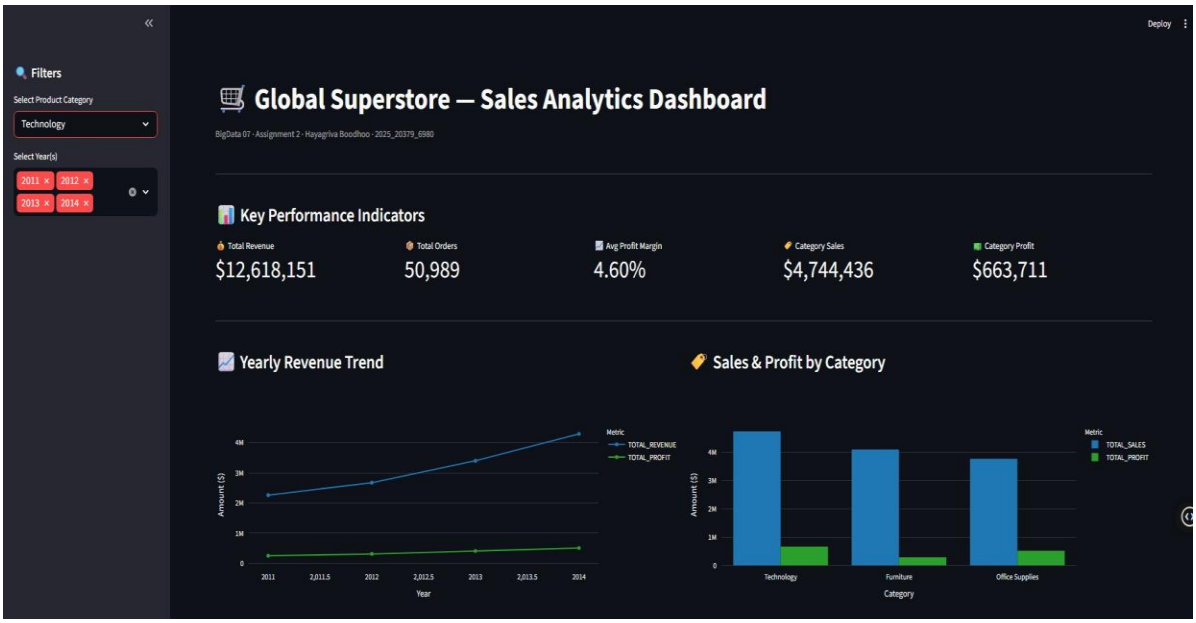


Figure 5.3 — Filter applied: Category = Technology. KPIs update reactively — Category Sales changes to $4,744,436 and Category Profit to $663,711

Screenshot — Technology Filter — Lower Section:

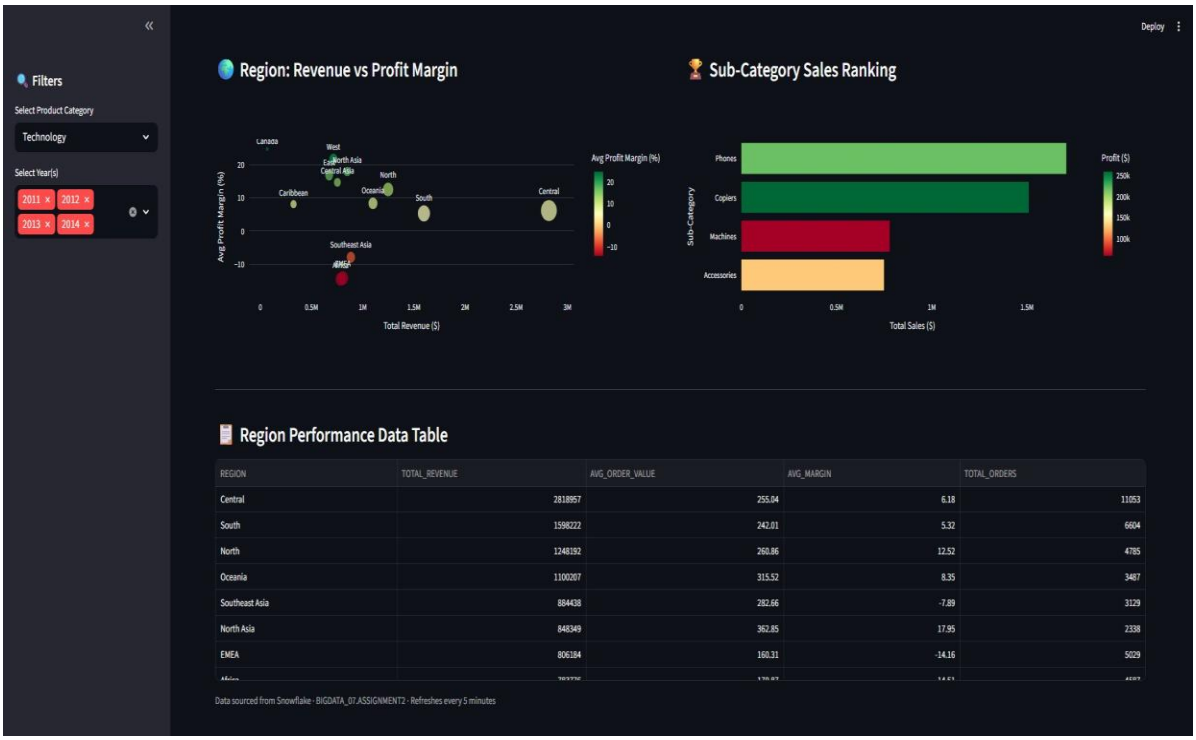


Figure 5.4 — Sub-category chart updates to show only Technology products (Phones, Copiers, Machines, Accessories) when Technology filter is applied

Hayagriva Boodhoo · 2025_20379_6980

Analysis Questions

Q1 — Patterns and Trends Observed

Several meaningful patterns emerged from the dashboard analytics:

- Revenue grew consistently year-over-year from $2.26M (2011) to $4.29M (2014) — a 90% increase over 4 years, indicating strong business expansion
- Technology leads in total revenue ($47.7M) but Office Supplies achieves a higher average profit margin (5.82% vs 4.95%), suggesting Technology high-volume sales are offset by higher costs or discounting
- The Tables sub-category is a significant value destroyer: $757K in sales but -$64K profit (-24.21% margin). This is the most actionable insight — management should restructure pricing or exit this product line
- Machines (-4.34% margin) and Appliances (-0.12% margin) also show negative returns, suggesting systemic pricing issues in the high-ticket physical goods segment
- The Central region dominates by total revenue ($2.82M, 22.34% share) but West achieves the highest profit margin (21.66%), indicating regional pricing discipline differences
- Southeast Asia, EMEA, and Africa all show negative average margins — three entire geographic markets destroying value and warranting strategic review

Q2 — Why InfluxDB/Snowflake over CSV or Relational Database

For this analytical pipeline, Snowflake is the appropriate choice for the following reasons:

- Columnar storage — Snowflake stores each column separately on disk. Analytical queries like AVG(Sales) read only the Sales column, not every field of every row. This delivers 10-100x faster query performance for aggregations
- Separation of compute and storage — Storage costs accrue only when data is at rest; compute (Virtual Warehouse) only activates during queries. For infrequent analytics workloads, this dramatically reduces cost
- Web UI for exploration — The Snowflake Worksheet enables ad-hoc SQL exploration without any local tooling, facilitating collaboration and iterative analysis
- Native Python connectivity — The snowflake-connector-python library enables direct DataFrame-to-table loading and parameterised queries from the Streamlit dashboard
- CSV files lack query capability, versioning, and concurrent access. A relational database (PostgreSQL) is row-oriented and optimised for transactional workloads (INSERT/UPDATE/DELETE), not analytical aggregations. Snowflake is purpose-built for analytics at scale

Q3 — Resilience to Kafka Broker Failure (Assignment 1 Context)

For the Assignment 1 Kafka pipeline, resilience to broker failure is achieved through:

- Replication factor ≥ 2 — Each partition is replicated across multiple brokers. If the leader broker fails, a follower is promoted automatically
- Consumer group offsets — Kafka tracks which messages each consumer has processed. After a failure, the consumer resumes from the last committed offset, avoiding data loss or duplication

Hayagriva Boodhoo · 2025_20379_6980

- min.insync.replicas configuration — Requires acknowledgement from a minimum number of replicas before confirming a write, preventing data loss even during simultaneous failures
- Producer retries with idempotent delivery — Enables exactly-once semantics so failed sends are retried without creating duplicate messages

Marking Scheme

Assessment Criterion	Max Marks	Score
Task 1 — Data loaded and cleaned, types cast, derived column adds value, quality report shown	5	
Task 2 — Four business questions answered correctly using both DataFrame API and Spark SQL	6	
Task 3 — Rank, cumulative total, and growth metric window functions all correct	5	
Task 4 — Data loaded into Snowflake; SQL queries run correctly and return valid results	6	
Task 5 — Dashboard queries Snowflake, filter is reactive, all visual elements populated	8	
TOTAL	30	

_____END OF ASSIGNMENT 2 REPORT _____

Hayagriva Boodhoo · 2025_20379_6980